

第3章 程序设计与算法思维

第5讲 程序构造：递归与迭代

邹华

武汉大学计算机学院

第3章 程序设计与算法思维

2

一、计算机语言的发展

二、不同的计算机语言之异同点

三、程序构造：控制结构

四、程序构造：函数、递归与迭代

五、算法表达与思维

计算系统与程序

3

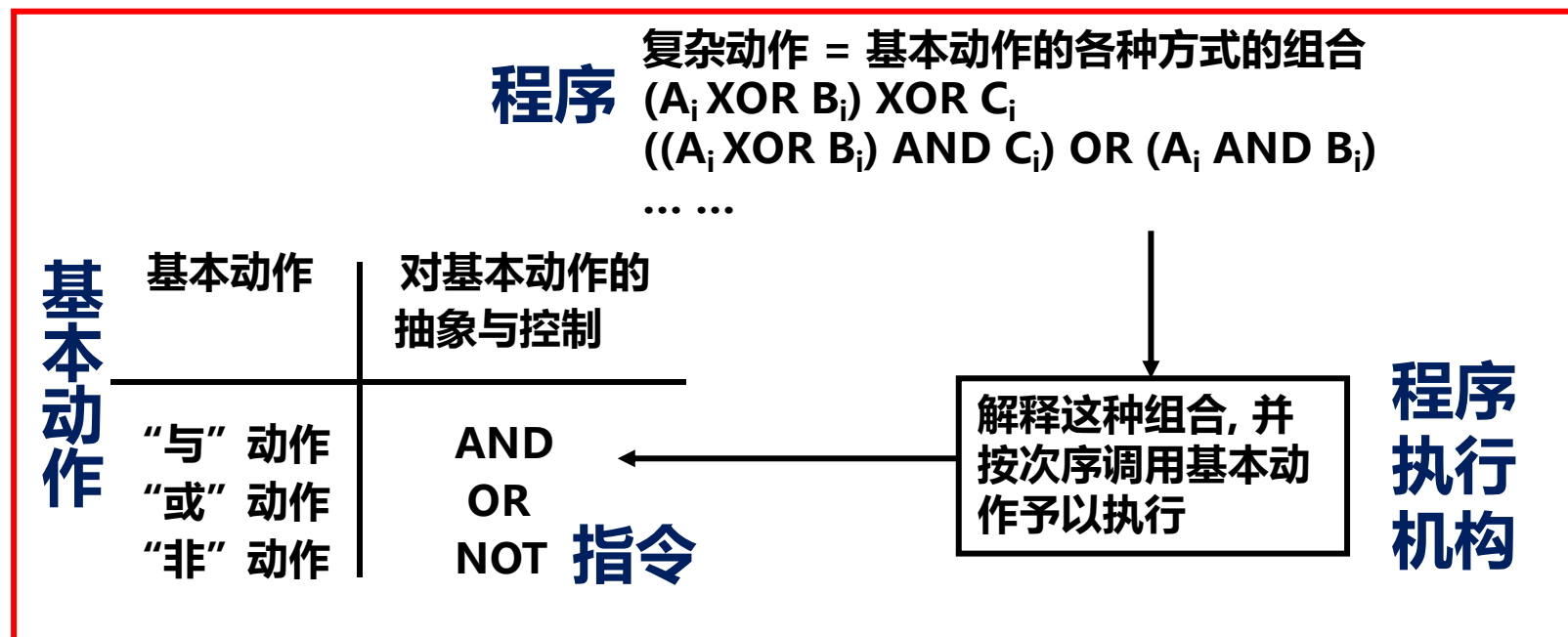
自动计算机器：程序与程序执行机构

计算系统 = 基本动作 + 指令 + 程序执行机构

指令 = 对可执行基本动作的抽象，即控制基本动作执行的命令

程序 = 基本动作指令的一个组合或执行序列, 用以实现复杂的动作

程序执行机构 = 负责解释程序即解释指令之间组合并按次序调用指令即调用基本动作执行的 机构



计算系统与程序

4

分层式构造

高抽象层次vs.低抽象层次

抽象： 将经常使用的、可由低层次系统实现的一些复杂动作，进行**命名**，以作为高层次系统的指令被使用

一种较高抽象层次的系统

基本动作

对基本动作的抽象与控制

“加” 动作
“减” 动作
“乘” 动作
“除” 动作

+
-
x
÷

指令

抽象

程序

复杂动作 = 基本动作的各种方式的组合
 $(V1 + V2) \times (V3 \div V4) \div V5$
 $(V1 \div (V2 \times (V3 + V4)) - (V5 \times V6))$
... ..

解释这种组合, 并按次序调用基本动作予以执行

程序执行机构

一种较低抽象层次的系统

基本动作

基本动作

对基本动作的抽象与控制

“与” 动作
“或” 动作
“非” 动作

AND
OR
NOT

指令

程序

复杂动作 = 基本动作的各种方式的组合
 $(A_i \text{ XOR } B_i) \text{ XOR } C_i$
 $((A_i \text{ XOR } B_i) \text{ AND } C_i) \text{ OR } (A_i \text{ AND } B_i)$
... ..

解释这种组合, 并按次序调用基本动作予以执行

程序执行机构

程序的作用和本质

什么是程序？
程序的本质是什么？



程序构造示例——利用运算组合式进行构造

6

运算组合式——一种程序的表达方法

由数值，到基本运算组合式

100

205

实际的数值

(100 + 205)

中缀表示法, 用运算符(即前述的指令)
将两个数值组合起来, 运算符在中间



(+ 100 205)

前缀表示法, 用运算符(即前述的指令)
将两个数值组合起来, 运算符在前面

将运算符表示的操作应用于后面的一组数值上, 求出结果

(+ 100 205 300 400 51 304)

一个运算符可以表示连加, 连减等情况

程序构造示例—利用运算组合式进行构造

7

运算组合式——一种程序的表达方法

由数值，到基本运算组合式

(+ 100 205)

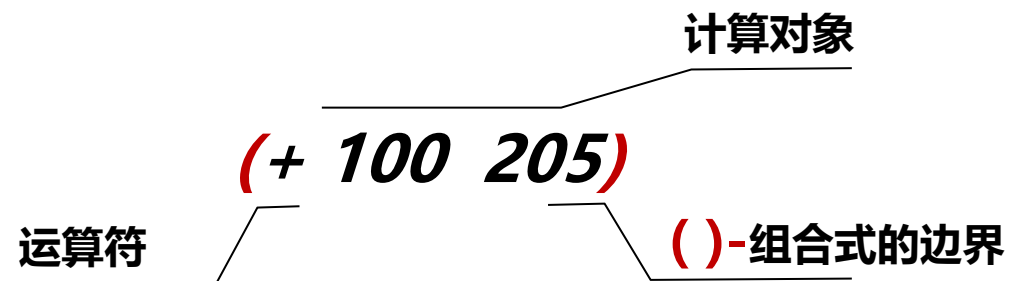
(- 200 50)

(200 5)*

(20 5 4 2)*

(- 20 5 4 2)

(+ 20 5 4 2)



- 一个组合式内只能有一个运算符，可多个计算对象
- 基本运算符只有+、-、*、/，可以被识别和计算。

一起练习,书写程序,

程序构造示例—利用运算组合式进行构造

8

运算组合式的“组合-嵌套”（构造）及其计算过程（执行）

$(+ 100 205)$

$(+ (+ 60 40) (- 305 100))$

(运算符1 计算对象1 计算对象2)

$(* (* 3 (+ (* 2 4) (+ 3 5))) (+ (- 10 7) 6))$

(运算符2 计算对象3 计算对象4)



计算过程

(运算符1 (运算符2 计算对象3 计算对象4) 计算对象2)

$(* (* 3 (+ (* 2 4) (+ 3 5))) (+ (- 10 7) 6))$

$(* (* 3 (+ 8 8)) (+ 3 6))$

$(* (* 3 16) 9)$

$(* 48 9)$

432

组合

嵌套

测练题：运算组合式 $(+ (+ 20\ 30\ 40) (*\ 2\ 5))$ 的结果是_____。

- ☐ A 900
- ☒ B 100
- ☐ C 97
- ☐ D 630
- ☐ E 前述没有正确答案

提交

程序构造示例—利用运算组合式进行构造

10

用名字替代计算对象（构造）

命名计算对象和构造中使用名字及计算中以计算对象替换名字

(define height 2) 名字的定义：定义名字height与2关联，
以后可以用height来表示2
一种类型的名字：数值型的名字

(+ (+ height 40) (- 305 height)) 名字的使用
(+ (50 height) (- 100 height))*

计算对象
(define height 2)
运算符

注意：不同类型的对象可以有不同的定义方法。这里统一用define来表示，在具体的程序设计语言中是用不同的方法来定义的

程序构造示例—利用运算组合式进行构造

11

用名字替代计算对象 (构造)

```
(define pi 3.14159)  
(define radius 10)  
(* pi (* radius radius))
```

```
(* pi (* radius radius))  
(* pi (* 10 10))  
(* pi 100)  
(* 3.14159 100)  
314.159 计算
```

```
(define circumference (* 2 pi radius))  
(* circumference 20)
```

```
(* circumference 20)  
(* (* 2 pi radius) 20)  
(* (* 2 3.14159 10) 20)  
(* 62.8318 20)  
1256.636 计算
```

测练题：已知下列运算组合式

(define **product** (+ (* 6 6) (* 8 8)))

(define **sumofproduct** (* product product))

(+ **sumofproduct** **sumofproduct**)

这段运算组合式最终的计算结果是_____。

A

20000

B

10000

C

100000000

D

400

E

前述没有正确答案

提交

程序构造示例—利用运算组合式进行构造

13

定义新运算符与使用新运算符（构造）

命名新运算符和构造中使用新运算符及执行中以过程替换新运算符

(define (square x) (x x))*

x^2

名字的定义：定义名字square为一个新的运算，即过程或称函数。另一种类型的名字：运算符型的名字

新运算符，即过程名或函数名

形式参数，
使用时将被实际参数所替代

过程体，用于表示新运算符的具体计算规则，其为关于形式参数x的一种计算组合。

(square 3)

(square 6)

名字的使用

计算对象2

(define (square x) (x x))*

运算符

计算对象1

注意：不同类型的对象可以有不同的定义方法。这里统一用define来表示，在具体的程序设计语言中是用不同的方法来定义的

程序构造示例—利用运算组合式进行构造

14

新运算符定义与构造示例

(square 10)

(square (+ 2 8))

(square (square 3))

(square (square (+ 2 5)))

(define (SumOfSquare x y) (+ (square x) (square y)))

(SumOfSquare 3 4)

(+ (SumOfSquare 3 4) height)

$$x^2 + y^2$$

程序构造示例—利用运算组合式进行构造

15

新运算符定义与构造示例

(define (NewProc a) (SumOfSquare (+ a 1) (a 2)))*

$(a+1)^2 + (a*2)^2$

(NewProc 3)

(NewProc (+ 3 1))

程序构造示例—利用运算组合式进行构造

16

运算组合式的执行 (计算方法1)

求值、代入、计算

(NewProc (+ 3 1))

(NewProc 4)

(SumOfSquare (+ 4 1) (4 2))*

(SumOfSquare 5 8)

(+ (Square 5) (Square 8))

(+ (5 5) (* 8 8))*

(+ 25 64)

89

先求值，再代入

程序构造示例—利用运算组合式进行构造

17

运算组合式的执行（计算方法2）

代入、求值、计算

代入到只有基本运算符时再开始求值

(NewProc (+ 3 1))

(SumOfSquare(+ (+ 3 1) 1) ((+ 3 1) 2))*

(+ (Square (+ (+ 3 1) 1)) (Square ((+ 3 1) 2)))*

(+ ((+ (+ 3 1) 1) (+ (+ 3 1) 1)) (* (* (+ 3 1) 2) (* (+ 3 1) 2)))*

(+ ((+ 4 1) (+ 4 1)) (* (* 4 2) (* 4 2)))*

(+ (5 5) (* 8 8))*

(+ 25 64)

89

先代入，
后求值

代入阶段
求值阶段

测练题：对于计算式，其正确的运算组合式(前缀表示法)为_____。

$$\frac{10 + \frac{20}{8 + 4}}{3 * 6 + 8 * 2}$$

- ☐ A $(/ (+ 10 / 20 + 8 4) (+ * 3 6 * 8 2))$
- ☐ B $((10 + (20 / (8 + 4))) / ((3 * 6) + (8 * 2)))$
- ☒ C $(/ (+ 10 (/ 20 (+ 8 4))) (+ (* 3 6) (* 8 2)))$
- ☐ D $(/ (/ 20 (+ 10 (+ 8 4))) (* (+ 3 6) (+ 8 2)))$
- ☐ E 前述没有正确答案

程序构造示例—利用运算组合式进行构造

19

构造与执行练习

对于计算式，其正确的运算组合式(前缀表示法)为_____。

- (A) $(/ (+ 10 / 20 + 8 4) (+ * 3 6 * 8 2));$
- (B) $((10 + (20 / (8 + 4))) / ((3 * 6) + (8 * 2)));$
- (C) $(/ (+ 10 (/ 20 (+ 8 4))) (+ (* 3 6) (* 8 2)));$
- (D) $(/ (/ 20 (+ 10 (+ 8 4))) (* (+ 3 6) (+ 8 2))).$

$$\frac{10 + \frac{20}{8 + 4}}{3 * 6 + 8 * 2}$$

(/ 操作数1 操作数2)

(/ (+ 10 操作数a2) (+ 操作数a3 操作数a4))

(/ (+ 10 (/ 20 (+ 8 4))) (+ 操作数a3 操作数
a4))

(/ (+ 10 (/ 20 (+ 8 4))) (+ (* 3 6) (* 8 2)))

测练题：请定义一个过程，求某一数值的立方 a^3

进一步以前述定义的过程，再定义一个过程，求某两个数值的立方和 a^3+b^3 。

进一步求 2^3+3^3 ，并模拟给出计算过程。

A

```
(define cube (* a a a))
(define sumofcube (+ cube cube))
(sumofcube 2 3)
35
```

B

```
(define (cube a) (* a a a))
(define (sumofcube a b) (* (cube a) (cube b)))
(sumofcube 2 3)
35
```

C

```
(define (cube x) (* a a a))
(define (sofcube x y) (+ (cube a) (cube b)))
(sofcube 2 3)
35
```

D

```
(define (func1 x) (* x x x))
(define (func2 x y) (+ (func1 x) (func1 y)))
(func2 2 3)
35
```

提交

什么是递归?

21

为什么需要递归?

怎样表达n!的计算过程

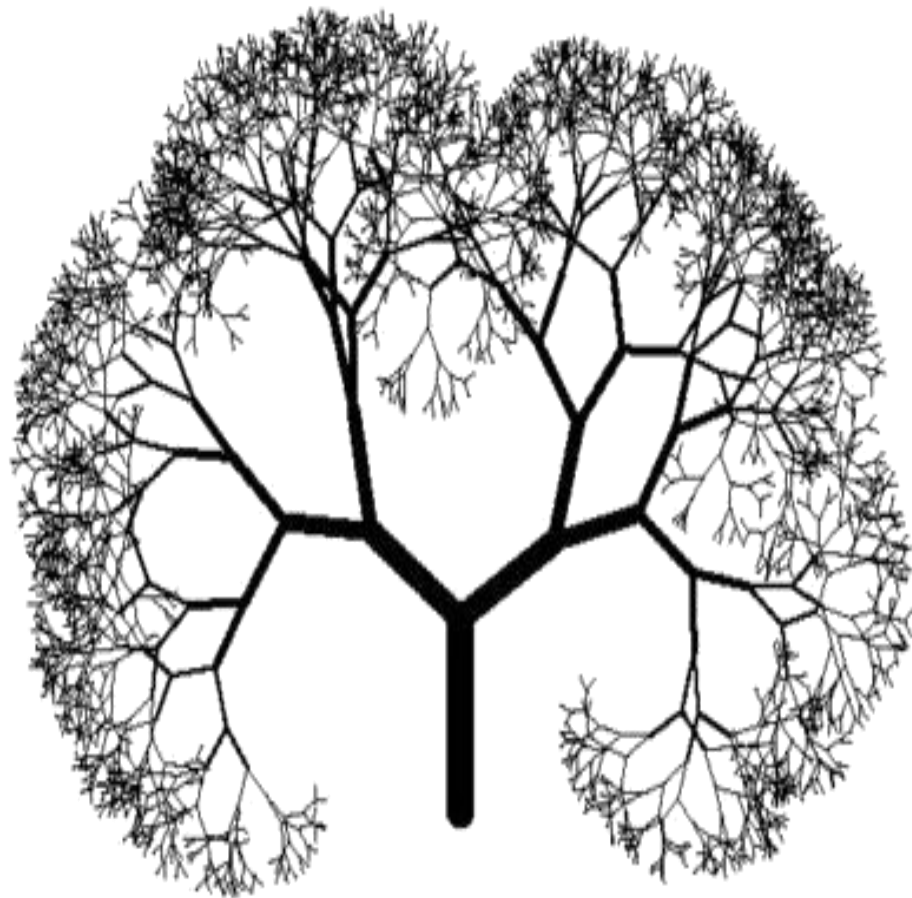
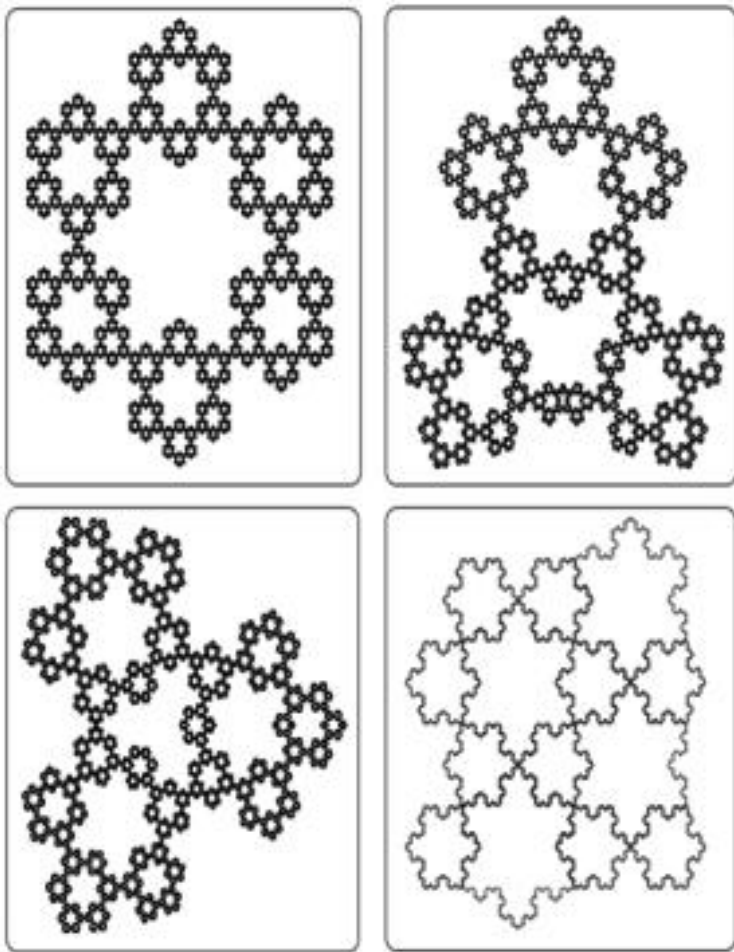
$(* \dots (* (* (* 1 \ 1) \ 2) \ 3) \dots n)$

怎样在表达中既去掉省略号，而又能表达近乎无限的内容?

什么是递归?

22

递归的典型视觉形式---自相似性事物的无限重复性构造



什么是递归?

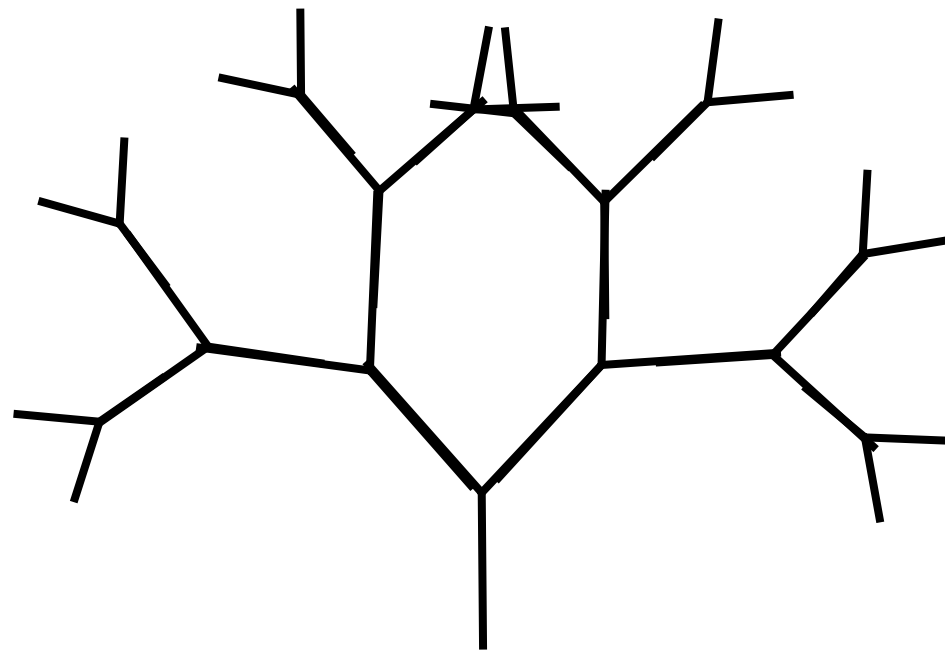
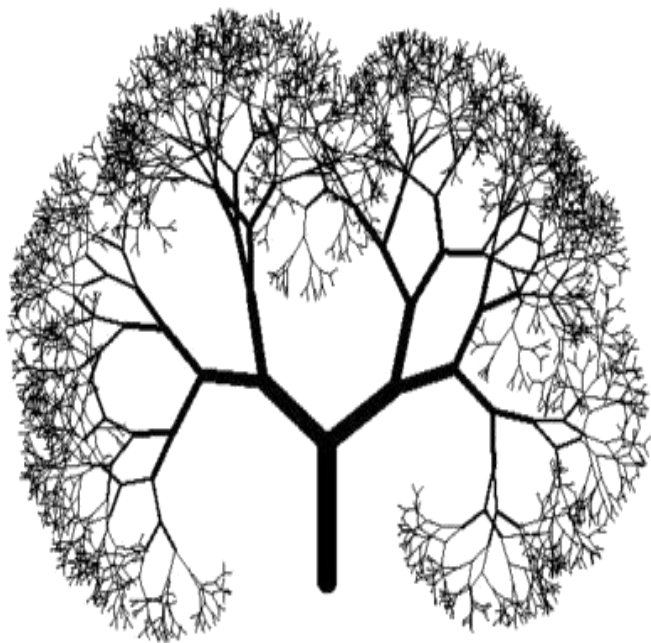
23

【基本概念】递归

递归是表达相似性对象及动作的无限性重复性构造的方法。

■ **递归基础**: 定义、构造和计算的起点, 直接给出。即 $h(0)$ 。

■ **递归步骤**: 由第 n 项或前 n 项定义第 $n+1$ 项。即: $h(n+1) = g(h(n), n)$, g 是需要明确给出的, 以说明 $h(n+1)$ 怎样由 $h(n)$ 和 n 构造出来。



什么是递归?

24

怎样递归?

【示例】怎样定义你的祖先?

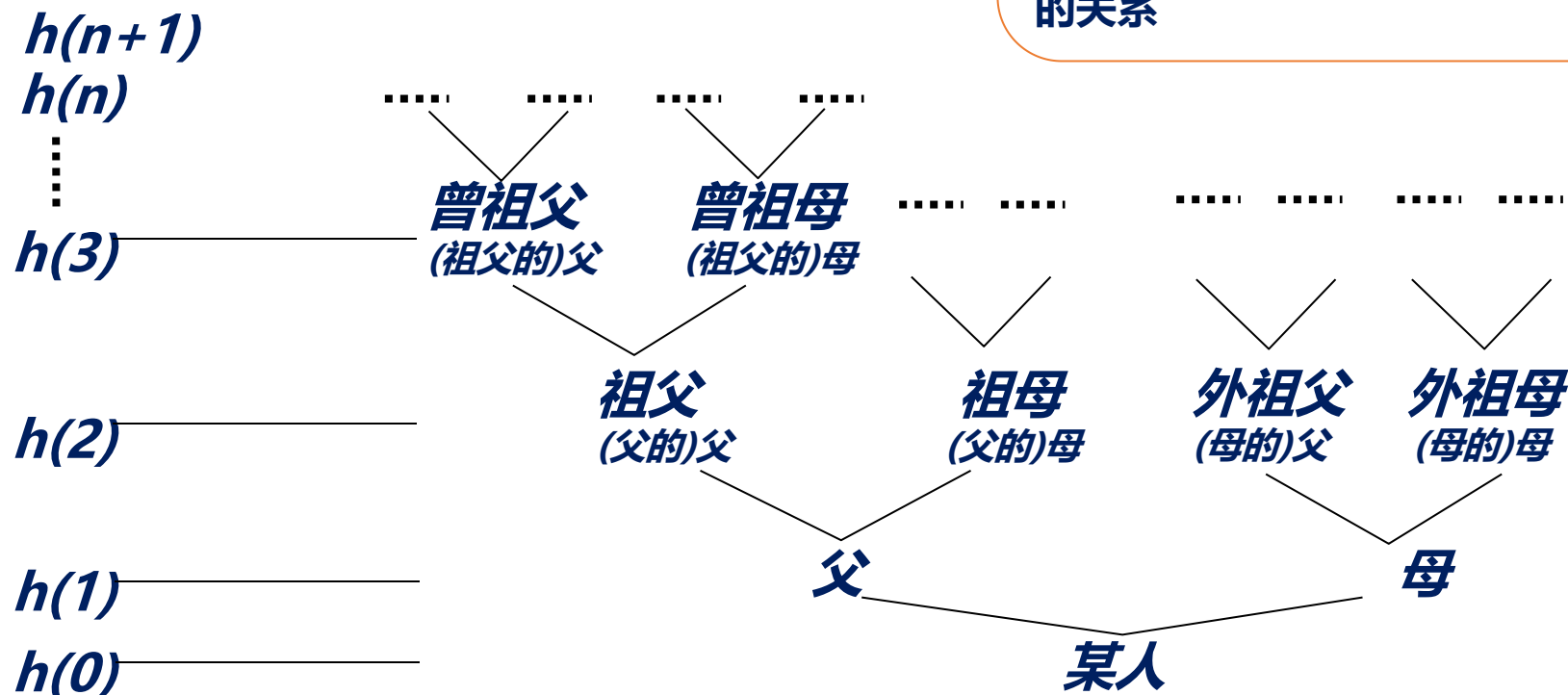
(1) 某人的双亲(父母)是他的祖先 (递归基础)

--- $h(1)$ 直接给出

(2) 某人祖先的双亲(父母)同样是某人的祖先 (递归步骤/公式)

--- $h(n+1) = g(h(n), n)$

--- n : 第几代; $h(n)$ 是第几代祖先; g 是 $h(n+1)$ 与 $h(n)$ 和 n 的关系



递归的理解
解比较难

递归思维是
计算机最重
要的特征

想学计算机
就要理解递
归思维

什么是递归?

25

数学的递推式

一个数列的第 n 项 a_n 与该数列的其他一项或多项之间存在某种对应关系，被表达为一种公式，称为**递推式**

等差数列递推公式

$$a_0 = 5$$

$$a_n = a_{n-1} + 3 \quad \text{当 } n \geq 1 \text{ 时}$$

等差数列的产生

$$a_0 = 5$$

$$a_1 = a_0 + 3 = 8$$

$$a_2 = a_1 + 3 = 11$$

$$a_3 = a_2 + 3 = 14$$

$$a_4 = a_3 + 3 = 17$$

... ..

- 第1项(或前 K 项)的值是已知的——**递推基础**;
- 由第 n 项或前 n 项计算第 $n+1$ 项——**递推规则/递推步骤**;
- 由前向后，可依次计算每一项

什么是递归?

26

为什么需要递归?

计算规则的构造与正确性判断

(运算符1 操作数1 操作数2)

(运算符2 操作数a 操作数b)

(运算符1 (运算符2 操作数a 操作数b) 操作数2)

(/ (+ 205 (/ (+ 15 3) (- 90 (× 8 8)))) (- 200 (× 10 5)))

(/ (+ 205 (/ (+ 15 3 - 90 (× 8 8)))) (- 200 (× 10 5)))

这个式子是有问题的哟



怎样表达运算组合式的层层构造--构造规则?

怎样判断一个运算组合式是否符合构造规则?

什么是递归?

27

运算组合式的递归定义示例

$(/ (+ 10 (- 8 (- 12 (+ 6 (/ 4 5))))) (- 15 8))$

是运算组合式吗?

(1)单一数值是运算组合式

--- $h(0)$ 直接给出, 递归基础

(2)如果 X, Y 是运算组合式, 则

$(+ X Y), (* X Y), (- X Y),$

$(/ X Y)$ 也是运算组合式

--- $h(n+1) = g(h(n), n)$, 即递归步骤

说明: 这里不允许连加、连减情况发生

--- n : 第几层; $h(n)$ 是第几层表达式;

g 是(运算符 操作数1 操作数2), 指出
 $h(n+1)$ 与 $h(n)$ 和 n 的关系

$h(n+1)$	_____	$(/ \text{ 操作数1 操作数2})$
$h(n)$	_____	$(/ (+ 10 \text{ 操作数a2}) \text{ 操作数2})$
$h(n-1)$	_____	$(/ (+ 10 (- 8 \text{ 操作数a3}) \text{ 操作数2})$
$\vdots$		
$h(1)$	_____	$(/ (+ 10 (- 8 (- 12 \text{ 操作数a4})) \text{ 操作数2})$
$h(0)$	_____	$(/ (+ 10 (- 8 (- 12 (+ 6 \text{ 操作数a5}))) \text{ 操作数2})$

$h(0)$	_____	$(/ 4 5)$
$h(1)$	_____	$(+ 6 (/ 4 5)$
$h(2)$	_____	$(- 12 (+ 6 (/ 4 5)))$
$\vdots$		
$h(n)$	_____	$(+ 10 (- 8 (- 12 (+ 6 (/ 4 5))))$
$h(n+1)$	_____	$(/ (+ 10 (- 8 (- 12 (+ 6 (/ 4 5)))) \text{ 操作数2})$

递归与迭代

一种类型的递归函数：定义是递归的, 但执行可以是递归的也可以是迭代的

□ Fibonacci数列, 无穷数列1, 1, 2, 3, 5, 8, 13, 21, 34, 55,,

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

递归定义

计算规则的表
达—告诉执行
者要递归计算

F(0)=1;

F(1)=1;

F(2)=F(1)+F(0)=2;

F(3)=F(2)+F(1)= 3;

F(4)=F(3)+F(2)= 3+2=5;... ..

递推计算/迭代计算/迭代执行

执行者要一步
一步地完成计
算获得结果

递归与迭代

29

另一种类型的递归函数：定义是递归的, 但执行也是递归的（用迭代不容易完成）

□ 阿克曼递归函数

$$A(m, n) = \begin{cases} n + 1 & \text{若 } m = 0 \\ A((m - 1), 1) & \text{若 } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{若 } m, n > 0 \end{cases}$$

$A(1, n) = ?$

测练题：

$$A(m, n) = \begin{cases} n+1 & \text{若 } m=0 \\ A((m-1), 1) & \text{若 } n=0 \\ A(m-1, A(m, n-1)) & \text{若 } m, n > 0 \end{cases}$$

$A(1, n) = ?$

- ☐ A $n+1$
- ☒ B $n+2$
- ☐ C $n+3$
- ☐ D 算不出来

提交

递归与迭代

31

另一种类型的递归函数：定义是递归的, 但执行也是递归的（用迭代不容易完成）

□ 阿克曼递归函数

$$A(m, n) = \begin{cases} n + 1 & \text{若 } m = 0 \\ A((m - 1), 1) & \text{若 } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{若 } m, n > 0 \end{cases}$$

$$\begin{aligned} A(1, n) &= A(0, A(1, n - 1)) \\ &= A(1, n - 1) + 1 \\ &= A(0, A(1, n - 2)) + 1 \\ &= A(1, n - 2) + 2 \\ &= A(1, n - 3) + 3 \\ &= \dots \\ &= A(1, n - n) + n \\ &= A(1, 0) + n \\ &= A(0, 1) + n \\ &= 2 + n \end{aligned}$$

$$\begin{aligned} A(1, n) &= A(0, A(1, n - 1)) \\ &= A(0, A(0, A(1, n - 2))) \\ &= A(0, A(0, A(0, A(1, n - 3)))) \\ &= \dots \\ &= A(0, A(0, \dots A(0, A(1, n - n)))) \\ &= A(0, A(0, \dots A(0, A(1, 0)))) \\ &= A(0, A(0, \dots A(0, A(0, 1)))) \\ &= A(0, A(0, \dots A(0, 2))) \\ &= A(0, A(0, \dots A(0, 3))) \\ &= A(0, n + 1) \\ &= n + 2 \end{aligned}$$

递归与迭代

32

另一种类型的递归函数：定义是递归的, 但执行也是递归的（用迭代不容易完成）

□ 阿克曼递归函数

$$A(m, n) = \begin{cases} n + 1 & \text{若 } m = 0 \\ A((m - 1), 1) & \text{若 } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{若 } m, n > 0 \end{cases}$$

递归定义

函数本身是递归的,
函数的变量也是递归的

计算规则的表
达—告诉执行
者要递归计算

$A(1, n) = A(0, A(1, n - 1)) = A(0, \langle \dots \text{代入前式计算过程} \rangle) = A(0, n + 1) = n + 2$ 。

$A(2, 1) = A(1, A(2, 0))$ --A(2,1)按m,n>0公式代入
= $A(1, A(1, 1))$ --A(2,0)按n=0公式代入
= $A(1, A(0, A(1, 0)))$ --A(1,1)按m,n>0公式代入
= $A(1, A(0, A(0, 1)))$ --A(1,0)按n=0公式代入
= $A(1, A(0, 2))$ --A(0,1)按m=0公式代入
= $A(1, 3)$ --A(0,2)按m=0公式代入
= $A(0, A(1, 2))$ --A(1,3)按m,n>0公式代入
= $A(0, A(0, A(1, 1)))$ --A(1,2)按m,n>0公式代入
= $A(0, A(0, A(0, A(1, 0))))$ --A(1,1)按m,n>0公式代入
= $A(0, A(0, A(0, A(0, 1))))$ --A(1,0)按n=0公式代入
= $A(0, A(0, A(0, 2)))$ --A(0,1)按m=0公式代入
= $A(0, A(0, 3)) = A(0, 4) = 5$ 。 --依次按m=0公式代入

执行者要一步
一步地完成计
算获得结果

递归计算/ 递归执行

由后向前代入,
再由前向后计算

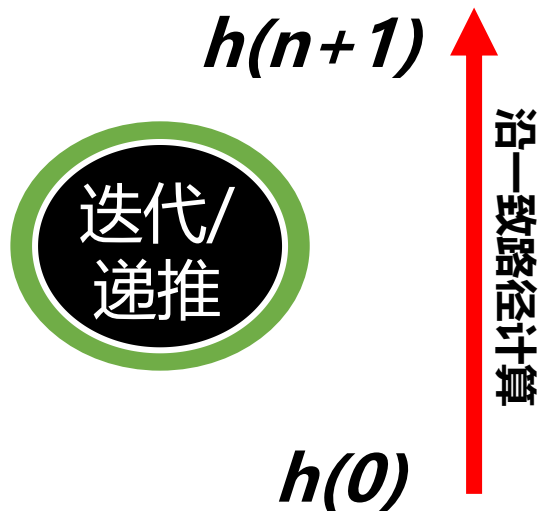
递归与迭代

33

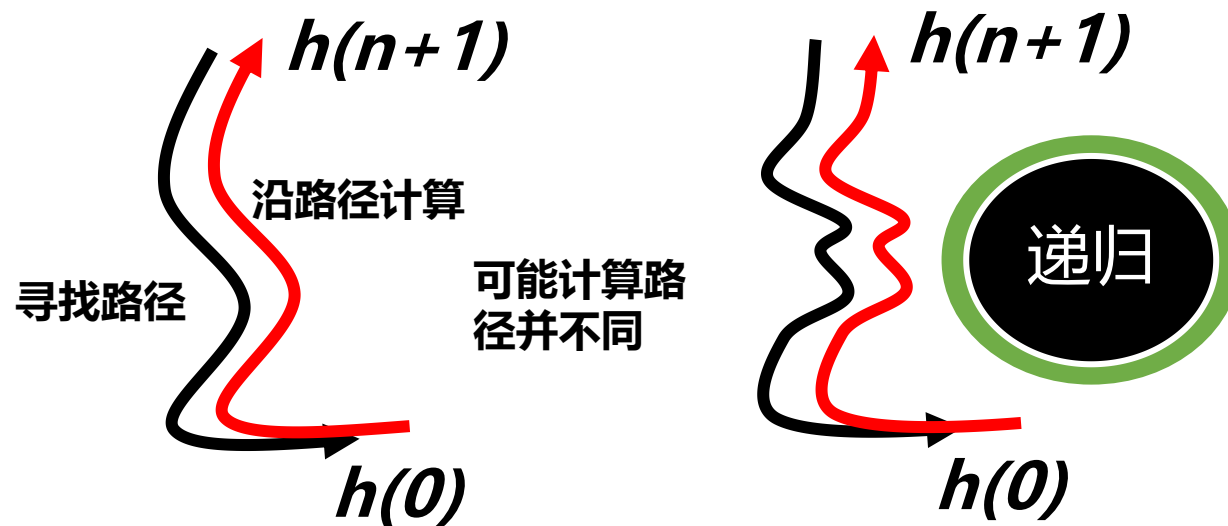
递归与迭代的差异

$$h(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ h(n-1) + h(n-2) & n > 1 \end{cases}$$

$$h(m, n) = \begin{cases} n + 1 & \text{若 } m = 0 \\ h((m-1), 1) & \text{若 } n = 0 \\ h(m-1, h(m, n-1)) & \text{若 } m, n > 0 \end{cases}$$



在前次结果基础上进行计算
可采用循环结构实现

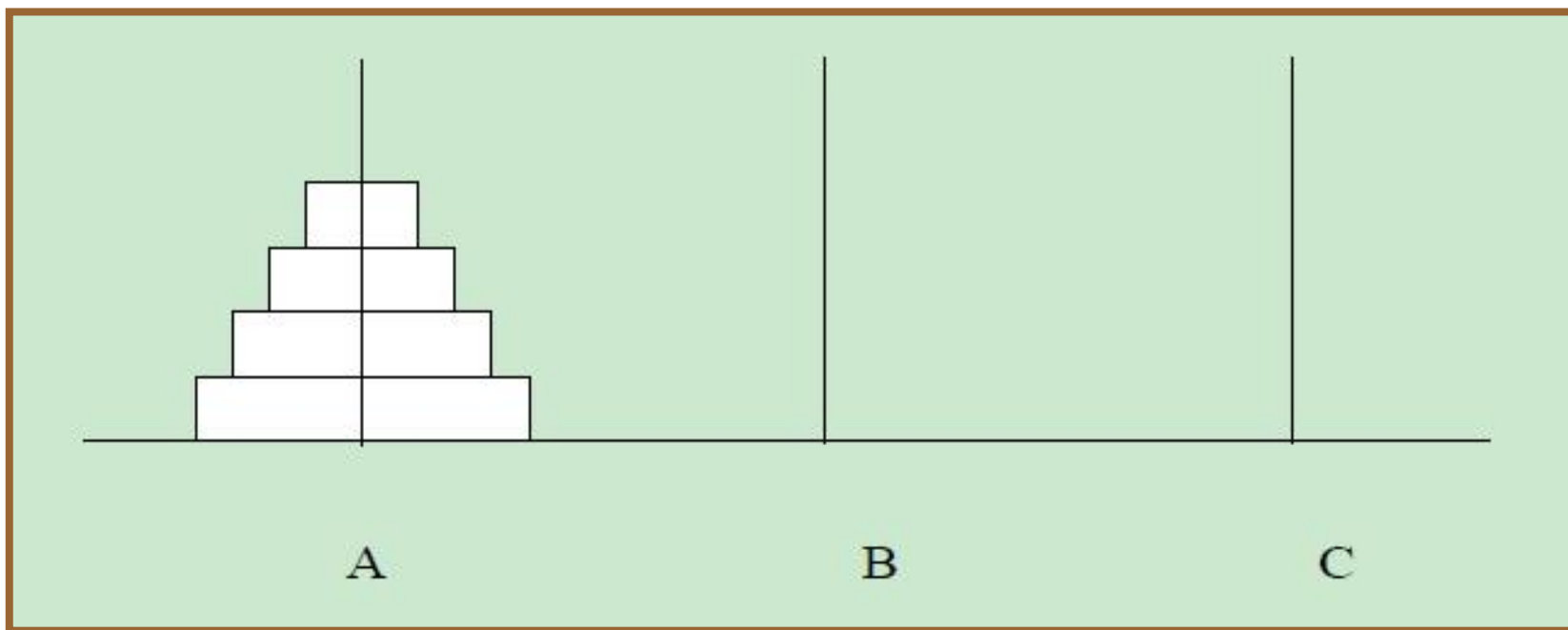


通常，只能由函数结构实现

递归与迭代

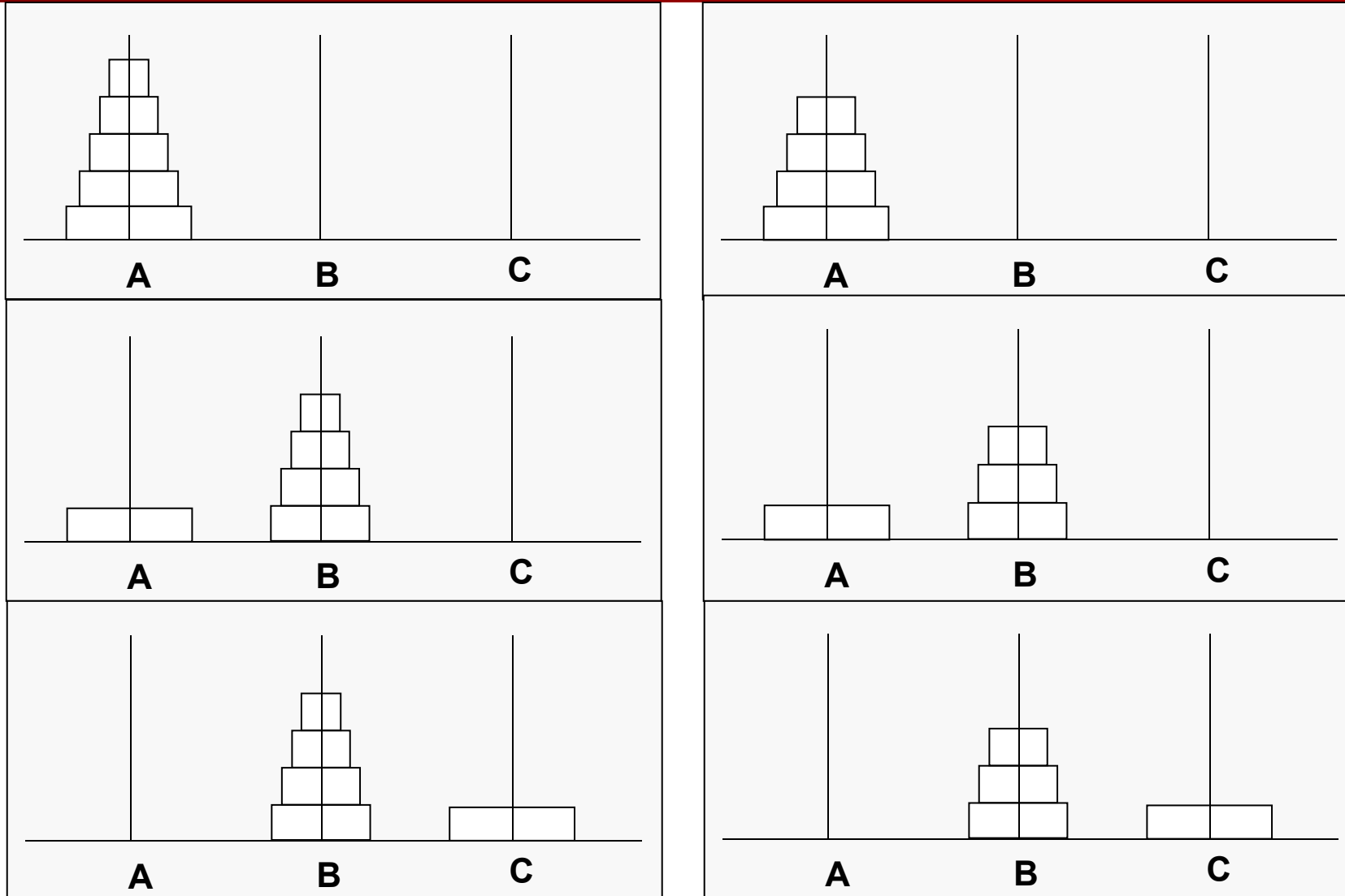
汉诺塔问题

梵天塔(汉诺塔)问题：有三根柱子，梵天将64个直径大小不一的金盘子按照从大到小的顺序依次套放在第一根柱子上形成一座金塔，要求每次只能移动一个盘子，盘子只能在三根柱子上来回移动不能放在他处，在移动过程中三根柱子上的盘子必须始终保持大盘在下小盘在上。



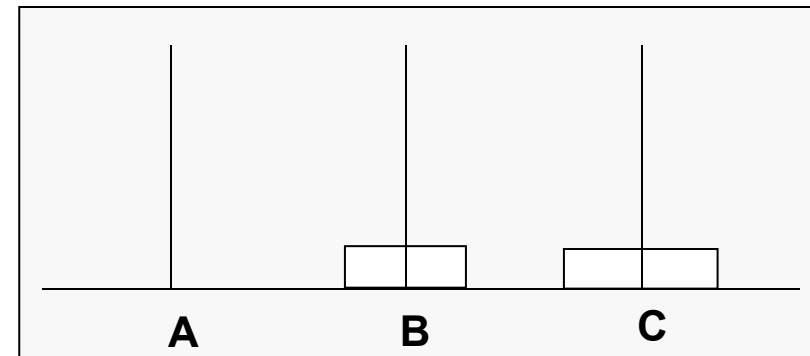
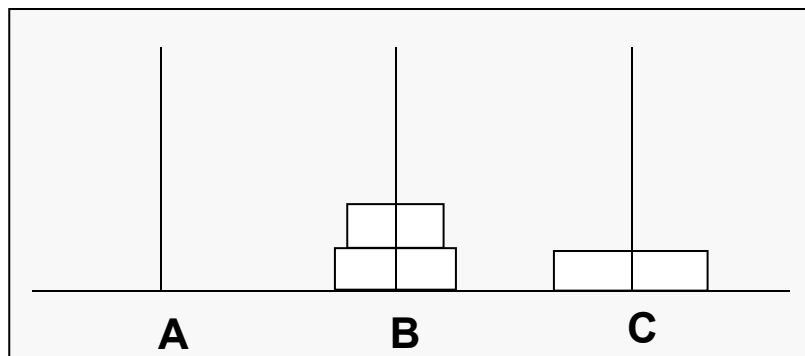
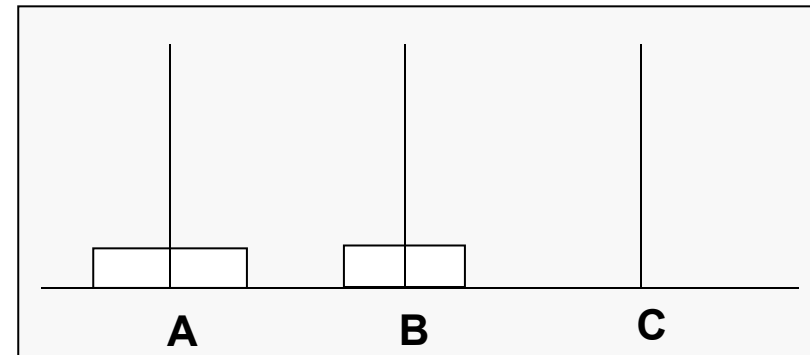
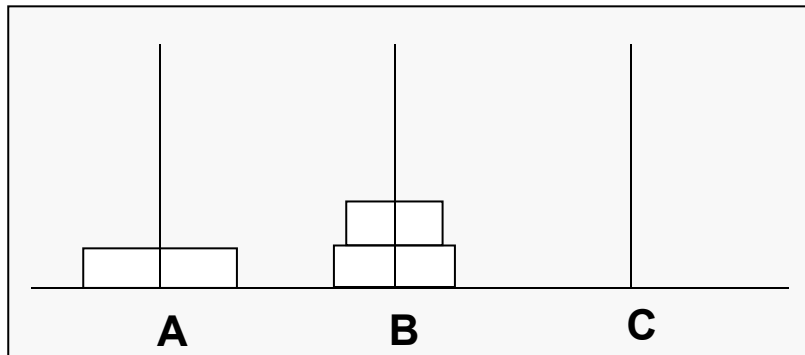
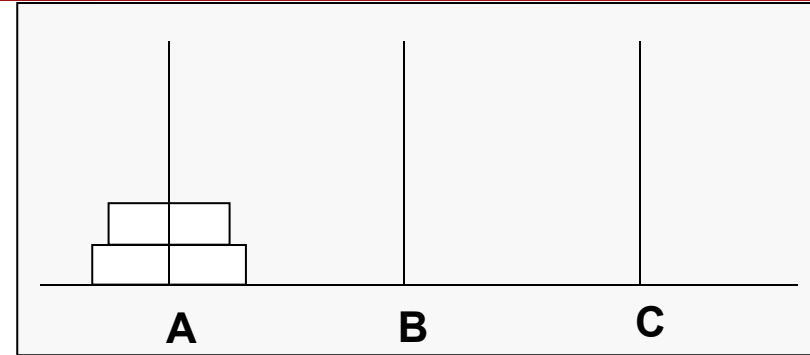
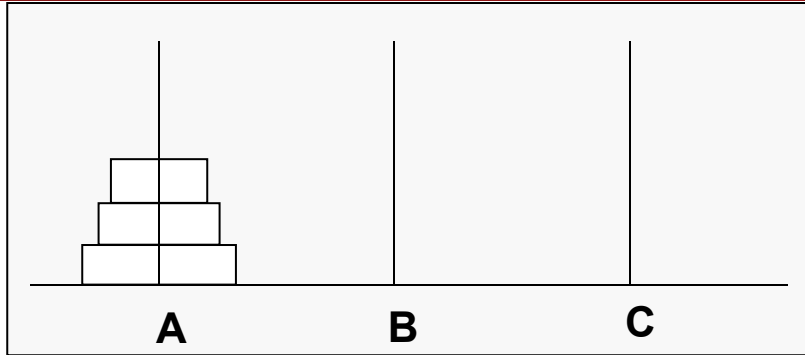
递归与迭代

汉诺塔问题求解示意



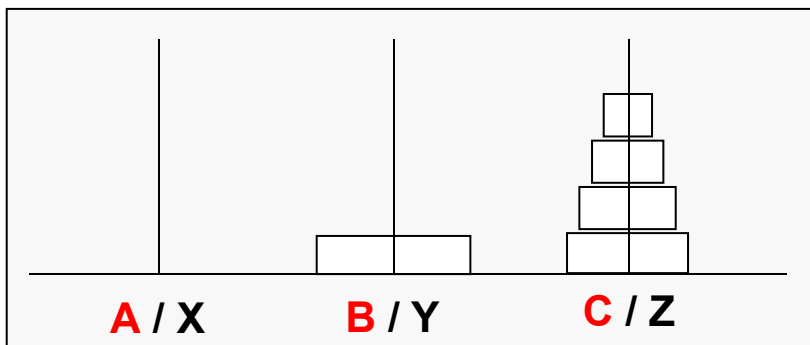
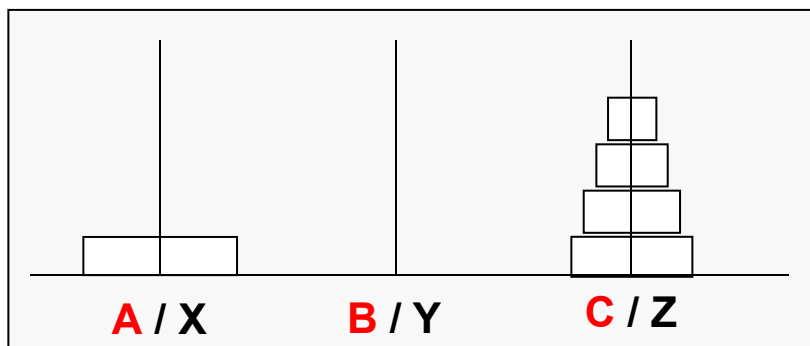
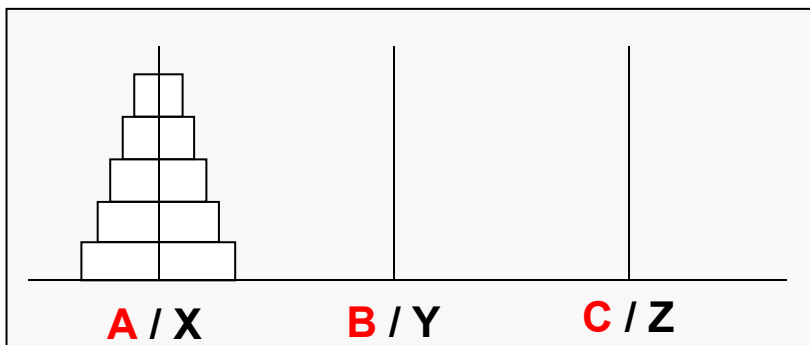
递归与迭代

汉诺塔问题求解示意



递归与迭代

汉诺塔问题求解示意



#假设有5个盘子的汉诺塔

```
result = Hanoi(5, "A", "B", "C")
print(result)
```

```
def Hanoi (N, X, Y, Z):
```

#该函数是将N个盘子从X柱，以Z柱做中转，移动到Y柱上

```
if N > 1:
```

#先把n-1个盘子从X放到Z上(以Y做中转)

```
Hanoi (N - 1, X, Z, Y)
```

#然后把X上最下面的盘子放到Y上

```
print(X, Y, sep='-->')
```

#接着把n-1个盘子从Z上放到Y上(以X做中转)

```
Hanoi (N - 1, Z, Y, X)
```

```
else:
```

#只有一个盘子时，直接把它从X放到Y上

```
print(X, Y, sep='-->')
```

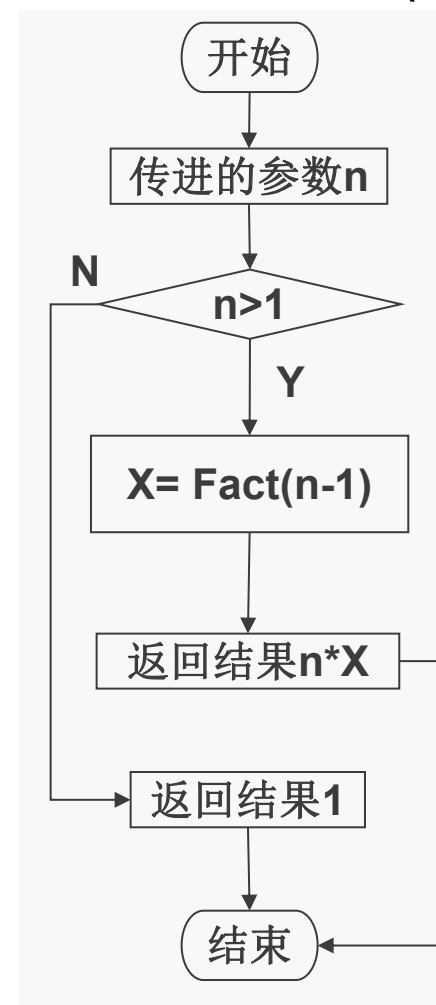
递归与迭代

递归法求n!的算法或程序

$$n! = \begin{cases} 1 & \text{当 } n \leq 1 \text{ 时} \\ n \times (n-1)! & \text{当 } n > 1 \text{ 时} \end{cases}$$

```
def Fact(n):  
    if n > 1:  
        #递归调用  
        x = Fact(n-1)  
        return n*x  
    else:  
        #递归基础  
        return 1
```

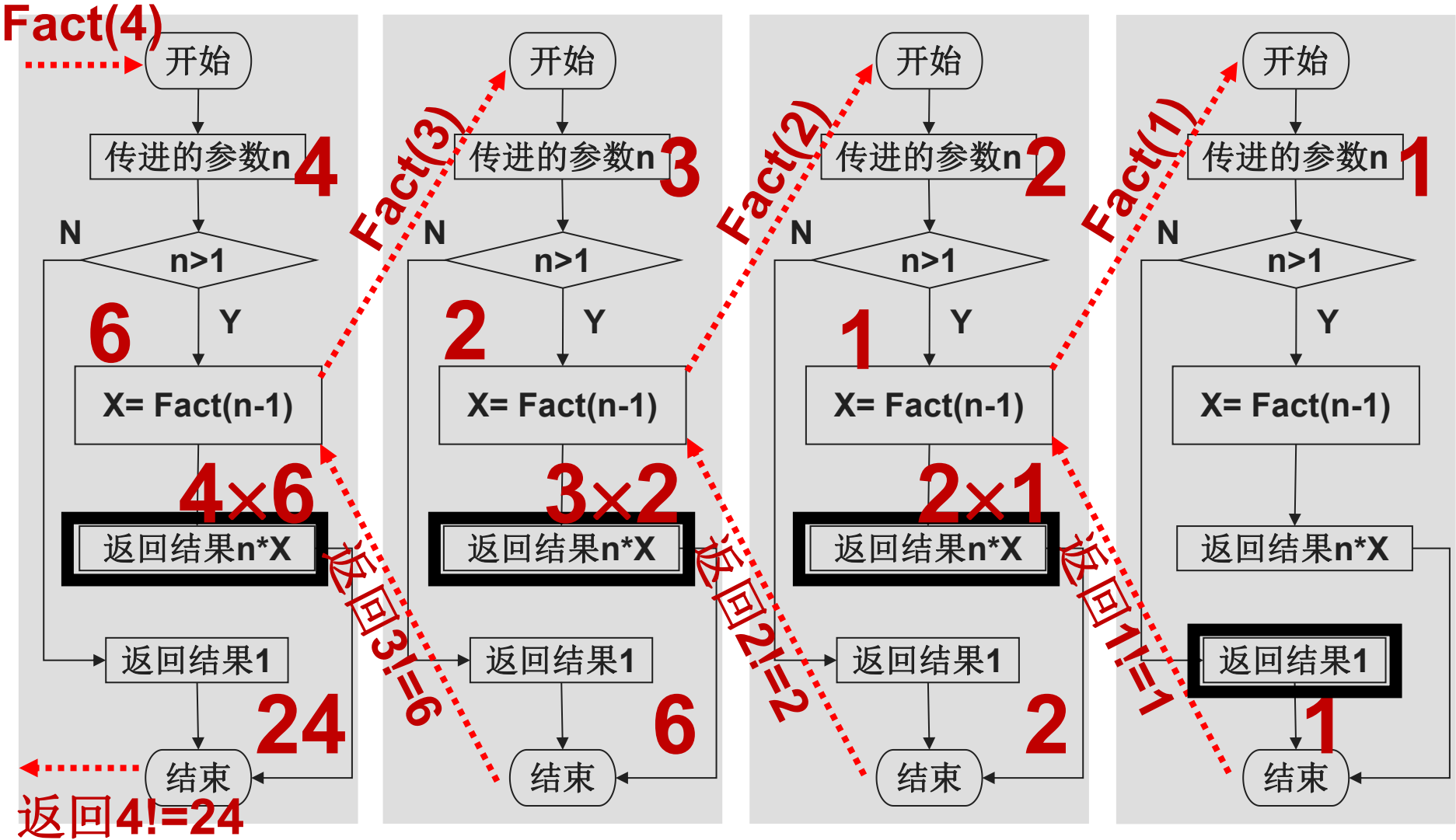
Fact(n)



递归与迭代

39

递归程序的执行过程：利用【堆栈】保存待计算的数据



1	Fact(4)的程 序断点
2	Fact(4)的程 序断点
3	Fact(4)的程 序断点
4	
5	
6	

堆栈：后进先出

递归与迭代

40

迭代程序的执行过程?

【示例】求n!的算法或程序--用迭代方法构造：循环-替代-递推

$$n! = \begin{cases} 1 & \text{当 } n \leq 1 \text{ 时} \\ 1 \times 2 \times \dots (n-1) \times n & \text{当 } n > 1 \text{ 时} \end{cases}$$

```
def Fact(n):  
    counter=1;  
    product=1;  
    #迭代  
    for counter in range(1, n+1):  
        product = product * counter  
    return product
```

Fact(5)的执行过程

	Counter	Product
初始值		1
循环第1次	1	1
循环第2次	2	2
循环第3次	3	6
循环第4次	4	24
循环第5次	5	120
退出循环	6	120

测练题：已知右侧程序， $n=4$ ，程序执行完时，`counter`，`product`的值分别是多少？

- ☐ A 4, 24
- ☒ B 5, 24
- ☐ C 5, 120
- ☐ D 6, 120

```
def Fact(n):
    counter=1;
    product=1;
    #迭代
    for counter in range(1, n+1):
        product = product * counter
    return product
```

提交

原始递归函数

42

【概念】原始递归函数

原始递归函数是接受自然数 x 或自然数的元组 $(x_1, \dots, x_n)$ 作为参数，并产生自然数的一个映射，记为 $f(x)$ 或 $f(x_1, \dots, x_n)$ 。接受 n 个参数的函数称作 **n 元函数**。处处有定义的函数被称作**全函数**，未必处处有定义的函数称作**半函数**或**部分函数**。

- **递归基础**：3个本原函数【**常数函数**、**后继函数**和**投影函数**】
- **递归步骤**：2个构造新函数的公式【**复合**--构造一个新函数、**原始递归**—构造系列新函数】

原始递归函数

43

递归基础：本原函数

最基本的原始递归函数，也被称为本原函数有三个：

- (1) **初始函数**：0元函数即常数无需计算；或者**常数函数**：对于每个自然数 n 和所有的 k , 有 $f(x_1, x_2, \dots, x_k) = n$ 。
- (2) **后继函数**：1元后继函数 S ，它接受一个参数并返回给出参数的**后继数**。例如 $S(1) = 2$, ..., $S(x) = x + 1$, 其中 x 为任意自然数。
- (3) **投影函数**：对于所有 $n \geq 1$ 和每个 $1 \leq i \leq n$ 的 i , n 元投影函数 P_i^n ，它接受 n 个参数并返回它们中的第 i 个参数，即

$$P_i^n(x_1, x_2, \dots, x_n) = x_i$$

原始递归函数

44

递归步骤--复合-构造一个新函数

(1)复合：给定原始递归函数 $f(x_1, \dots, x_k)$ ，和 k 个原始递归函数 $g_1, \dots, g_k$ ，则 f 和 $g_1, \dots, g_k$ 的复合是函数 h ，即

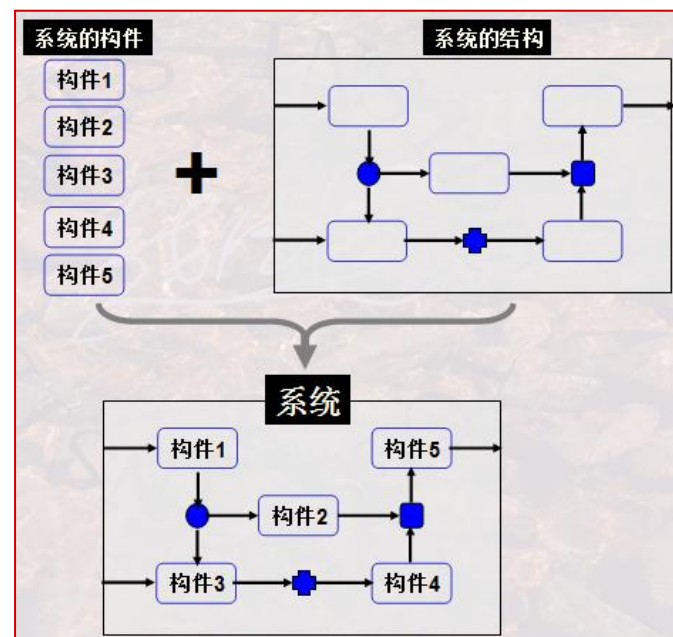
$$h(x_1, \dots, x_m) = f(g_1(x_1, \dots, x_m), \dots, g_k(x_1, \dots, x_m))$$

简单而言，复合是将一系列函数作为参数代入到另一个函数中，又被称为代入。复合是构造新函数的一种方法。复合是表达组合的一种方法。

$g_1, \dots, g_k$ 的指令组合关系 f vs. 基本指令 $g_1, \dots, g_k$

$g_1, \dots, g_k$ 的组合关系 f vs. 运算组合式 $g_1, \dots, g_k$

结构 f vs. 构件 $g_1, \dots, g_k$



原始递归函数

45

递归步骤—原始递归-构造系列新函数

(2)原始递归：给定原始递归函数 f 和 g ，则新函数 h 可由 f 和 g 递归的定义，其中

$$h(0, x_1, \dots, x_k) = f(x_1, \dots, x_k)$$

$$h(S(n), x_1, \dots, x_k) = g(h(n, x_1, \dots, x_k), n, x_1, \dots, x_k)$$

简单而言，定义新函数 h ，就是要定义 $h(0), h(1), \dots, h(n), \dots$ 。 $h(0)$ 直接给出。 $h(n+1)$ 则由将 $h(n)$ 和 n 代入 g 中来构造。

原始递归是递归地构造新函数的方法，尤其是无限的相似性函数的构造方法。

$$(* (* \dots (* (* (* 1 1) 2) 3) \dots n) S(n))$$

$$g(x_1, x_2) = (* x_1 S(x_2))$$

$$h(0) = 1$$

$$h(S(n)) = g(h(n), n)$$

$$h(0) = 1$$

$$h(1) = g(h(0), 0) = (* 1 1)$$

$$h(2) = g(h(1), 1) = (* (* 1 1) 2)$$

$$h(3) = g(h(2), 2) = ((* (* 1 1) 2) 3)$$

...

$$h(S(n)) = g(h(n), n)$$

原始递归函数

46

原始递归函数构造示例

已知: $f(x)=x$

$g(x_1,x_2,x_3)=x_1+x_2+x_3$, 其中 x,x_1,x_2,x_3 均为自然数

$h(0,x) = f(x)$ 且 $h(S(n), x) = g(h(n,x),n,x)$

该函数对任一自然数的计算过程为:

$$h(0,x)=f(x) =x$$

$$h(1,x)=h(S(0),x) = g(h(0,x),0,x) = g(f(x),0,x) = f(x) +0+ x =2x$$

$$h(2,x)=h(S(1),x) = g(h(1,x),1,x)=g(g(f(x),0,x),1,x)=g(2x, 1, x)=3x+1$$

$$\begin{aligned} h(3,x)=h(S(2),x)&=g(h(2,x),2,x)=g(g(h(1,x),1,x),2,x)=g(g(g(h(0,x),0,x),1,x),2,x) \\ &= \dots = 4x+3 \end{aligned}$$

... ..

原始递归函数

47

原始递归函数构造示例

已知: $f(x)=2$

$g(x_1, x_2, x_3)=x_1$, 其中 x, x_1, x_2, x_3 均为自然数

$h(0, x) = f(x)$ 且 $h(S(n), x) = g(h(n, x), n, x)$

该函数对任一自然数的计算过程为:

$$h(0, x) = f(x) = 2$$

$$h(1, x) = h(S(0), x) = g(h(0, x), 0, x) = g(f(x), 0, x) = f(x) = 2$$

$$h(2, x) = h(S(1), x) = g(h(1, x), 1, x) = g(g(f(x), 0, x), 1, x) = g(2, 1, x) = 2$$

$$\begin{aligned} h(3, x) &= h(S(2), x) = g(h(2, x), 2, x) = g(g(h(1, x), 1, x), 2, x) = g(g(g(h(0, x), 0, x), 1, x), 2, x) \\ &= \dots = 2 \end{aligned}$$

... ..

程序与递归

48

关于递归的理解

递归是表达相似性对象及动作的无限性重复性构造的方法。

■ **递归基础**：定义、构造和计算的起点，直接给出。即 $h(0)$ 。

■ **递归步骤**：由前 n 项或第 n 项定义第 $n+1$ 项；由低阶 $h(k)$ 且 $k < n$ ，来构造高阶 $h(n+1)$ ；

即： $h(n+1) = g(h(n), n)$ ，或者 $h(n+1) = g(h(k), n, k)$ 形式， g 是需要明确给出的，以说明 $h(n+1)$ 怎样由 $h(k)$ ， k 和 n 构造出来。

- 递归是一种关于抽象的表达方法---用递归定义无限的相似事物
- 递归是一种算法或程序的构造技术---自身调用自身，高阶调用低阶，构造无限的计算步骤
- 递归是一种典型的计算/执行过程---由后向前代入，直至代入到递归基础，再由递归基础向后计算直至计算出最终结果，即由前向后计算

用递归
定义

用递归
构造

递归计算
/执行

程序与递归

程序构造思维小结

什么是程序？ 程序的本质是什么？

概念/原理与案例相结合
知识伴随思维，思维贯通知识
相互关联且递进的方式展开与贯通

递归定义、递归算法、递归计算

程序构造的基本方法：递归与迭代

组合/抽象 → 递归

实例层面：运算组合式

概念层面：计算系统与程序

程序本质---组合、抽象、构造与执行

程序---对基本动作的组合

计算系统---执行程序的系统

计算系统的构造

递归函数又被称为可
计算函数。递归理论
也被称为可计算理论。
是计算机类学科理论
研究的基础

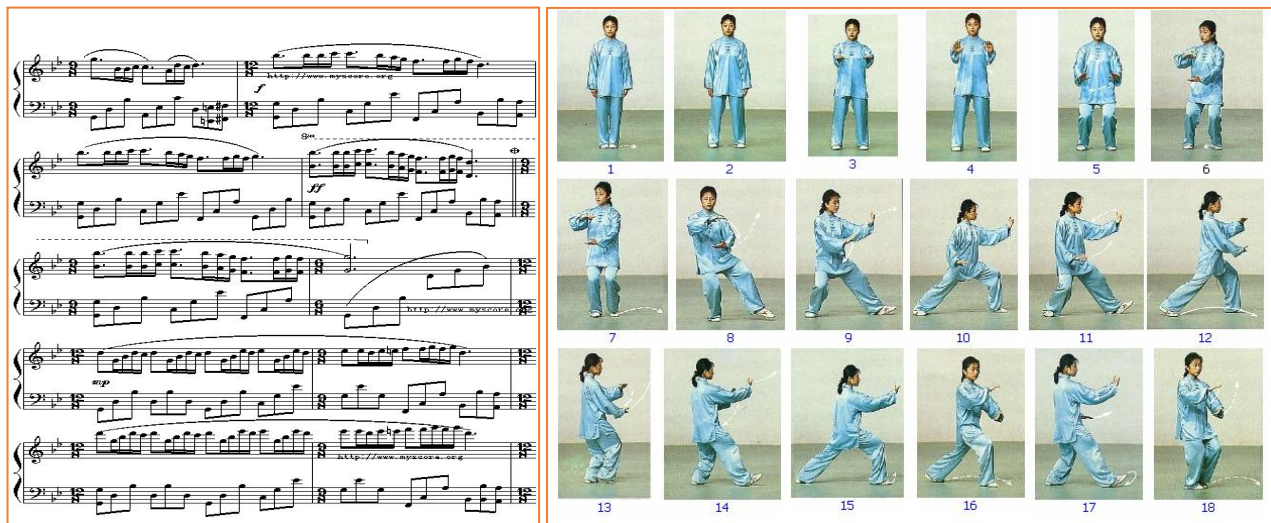
第3章 程序设计与算法思维

50

- 一、计算机语言的发展
- 二、不同的计算机语言之异同点
- 三、程序构造：控制结构
- 四、程序构造：函数、递归与迭代
- 五、算法表达与思维**

什么是算法？

- **算法---计算机与软件的灵魂。** “算法” (Algorithm)一词源于阿拉伯数学家阿科瓦里茨米 (AlKhowarizmi)，其在公元825年写了著名的《波斯教科书》 (Persian Textbook)，书中概括了进行四则算术运算的计算规则。
- **算法是一个有穷规则的集合**，它用规则规定了解决某一特定类型问题的运算序列，或者规定了任务执行或问题求解的一系列步骤。



如音乐乐谱、太极拳谱等都可看作广义的算法

算法的五个特征

- **有穷性**：一个算法在执行有穷步规则之后必须结束。
- **确定性**：算法的每一个步骤必须要确切地定义，不得有歧义性。
- **有限输入**：算法有零个或多个的输入。
- **有限输出**：算法有一个或多个的输出/结果，即与输入有某个特定关系的量。
- **能行性**：算法中有待执行的运算和操作必须是相当基本的(可以由机器自动完成)，并能在有限时间内完成。

寻找两个正整数的最大公约数的欧几里德算法

输入：正整数 M 和正整数 N

输出： M 和 N 的最大公约数(设 $M > N$)

算法步骤：

Step 1. M 除以 N , 记余数为 R

Step 2. 如果 R 不是 0 , 将 N 的值赋给 M , R 的值赋给 N , 返回Step 1; 否则, 最大公约数是 N , 输出 N , 算法结束。

基本运算：除法、赋值、逻辑判断

典型的“重复/循环”与“迭代”

算法的类自然语言表达方法

算法与程序的基本控制结构

- ✓ **顺序结构**: “**执行A, 然后执行B**”, 是按顺序执行一条条规则的一种结构。
- ✓ **分支结构**: “**如果Q成立, 那么执行A, 否则执行B**”, Q是某些逻辑条件, 即按条件判断结果决定执行哪些规则的一种结构。
- ✓ **循环结构**: 控制指令或规则的多次执行的一种结构---迭代(iteration)
 - ◆ 循环结构又分为有界循环结构和条件循环结构。
- ✓ **有界循环**: “**执行A指令N次**”, 其中N是一个整数。
- ✓ **条件循环**: 某些时候称为无界循环, “**重复执行A直到条件Q成立**” 或 “**当Q成立时反复执行A**”, 其中Q是条件。

算法表达示例

【示例】 $sum=1+2+3+4+\dots+n$ 求和问题的算法描述

Start of the algorithm(算法开始)

- (1)输入N的值;
- (2)设 i 的值为1; sum 的值为0;
- (3)如果 $i \leq N$, 则执行第(4)步, 否则转到第(7)步执行;
- (4)计算 $sum + i$, 并将结果赋给 sum ;
- (5)计算 $i+1$, 并将结果赋给 i ;
- (6)返回到第3步继续执行;
- (7)输出 sum 的结果。

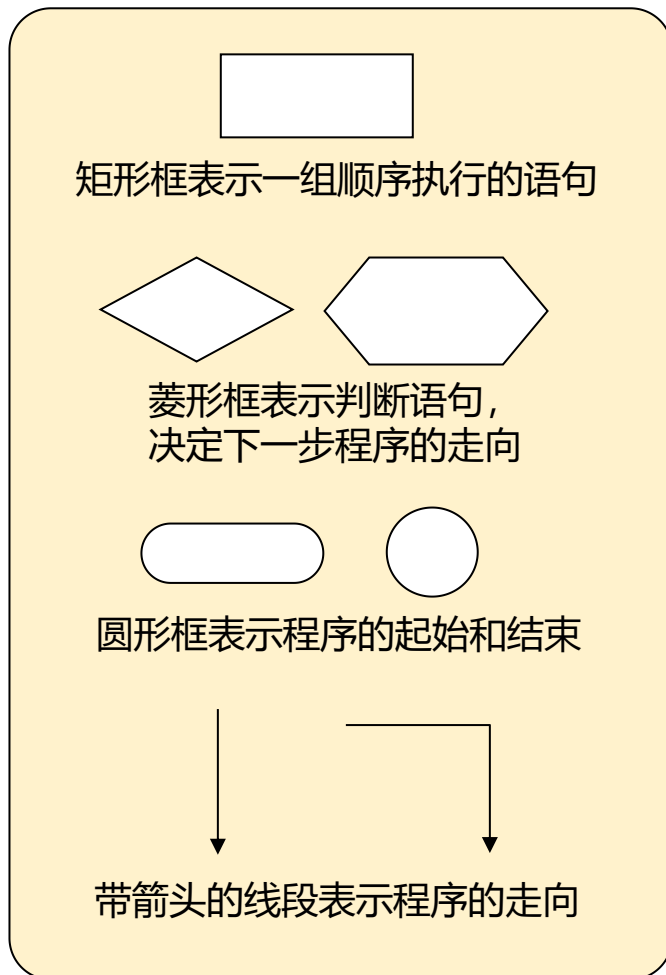
End of the algorithm(算法结束)

自然语言表示的算法容易出现二义性、不确定性等问题

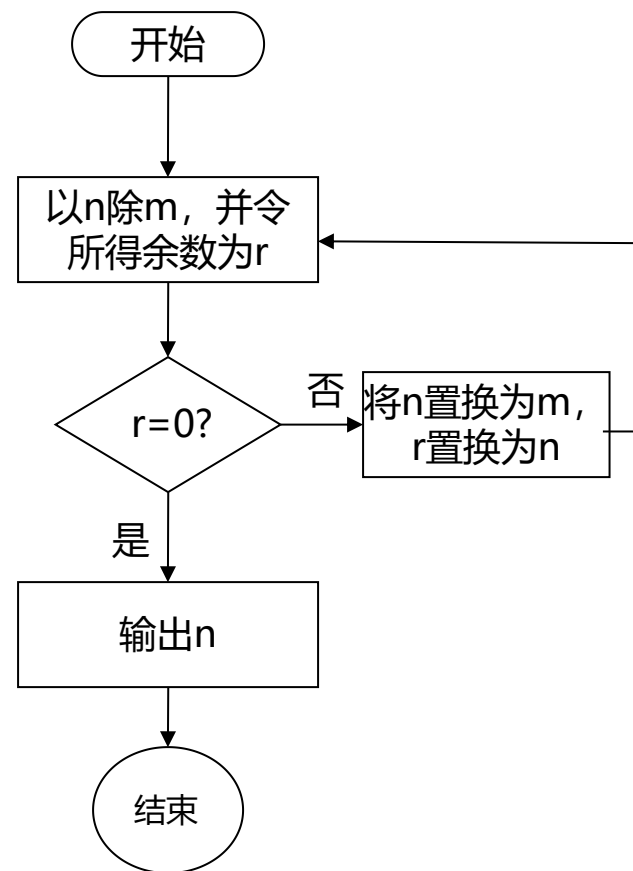
算法的程序流程图表达方法

流程图的基本表示符号

- ✓ **矩形框**: 表示一组顺序执行的规则或者程序语句。
- ✓ **菱形框**: 表示条件判断, 并根据判断结果执行不同的分支。
- ✓ **圆形框**: 表示算法或程序的开始或结束。
- ✓ **箭头线**: 表示算法或程序的走向。

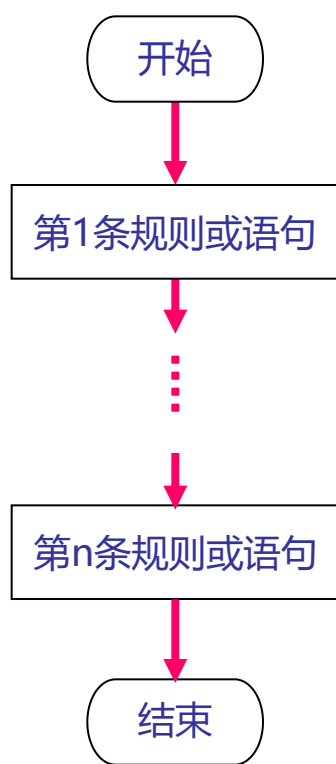


【示例】欧几里德算法流程图

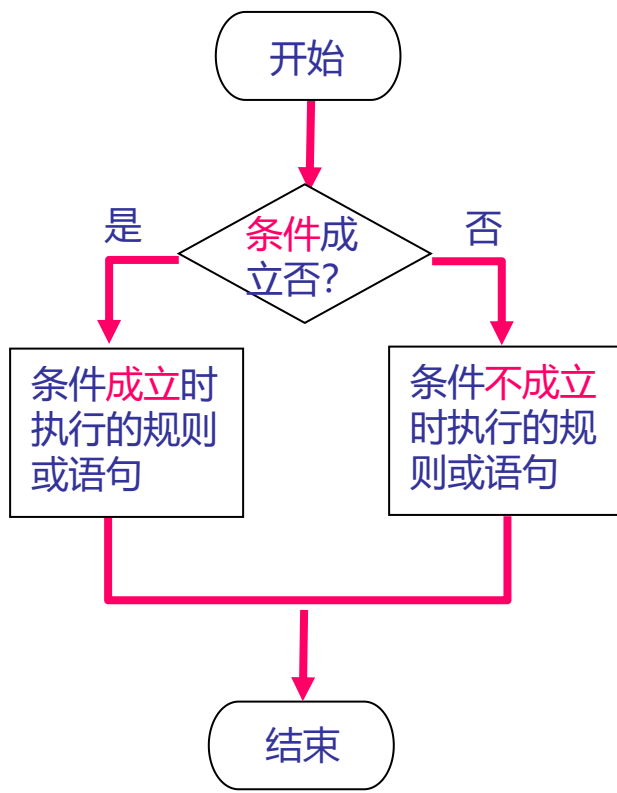


算法的程序流程图表达方法

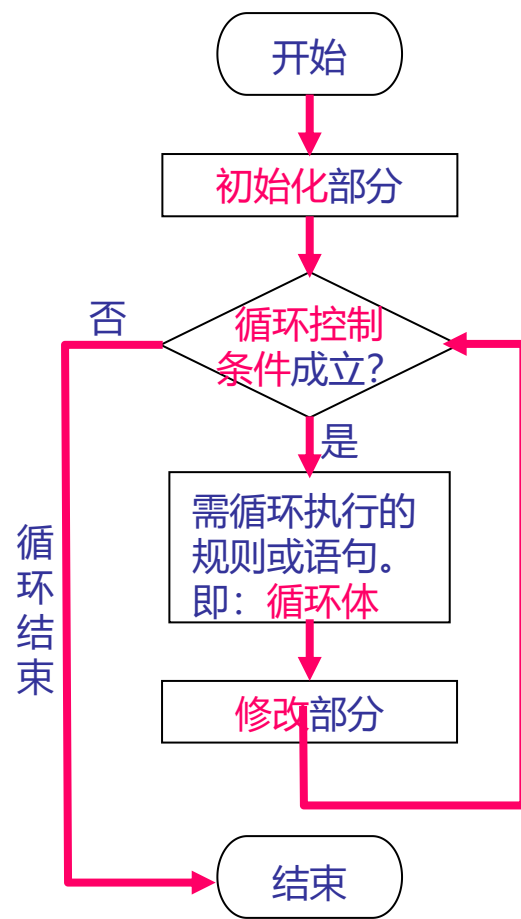
三种控制结构的流程图表达



顺序结构的流程图



分支结构的流程图



循环结构的流程图

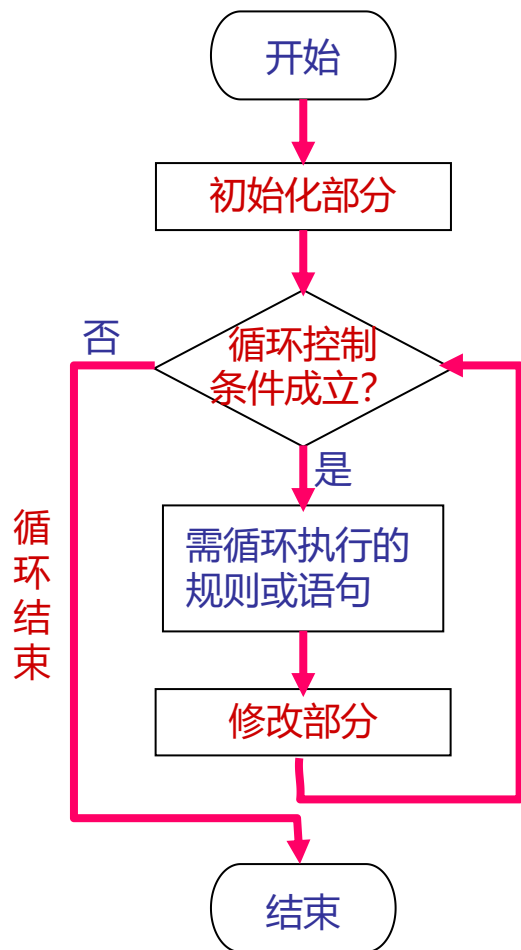
算法表达与思维

57

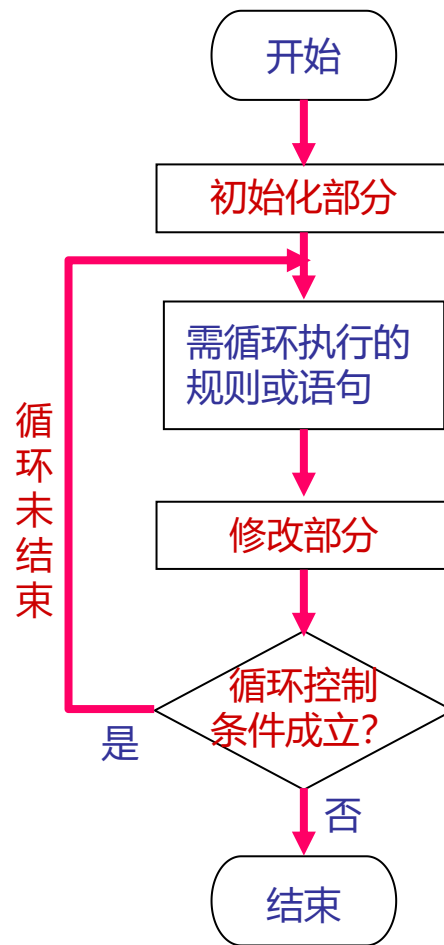
算法的程序流程图表达方法

循环结构的两种情况的流程图表达

有界循环结构的流程图
(循环次数确定)

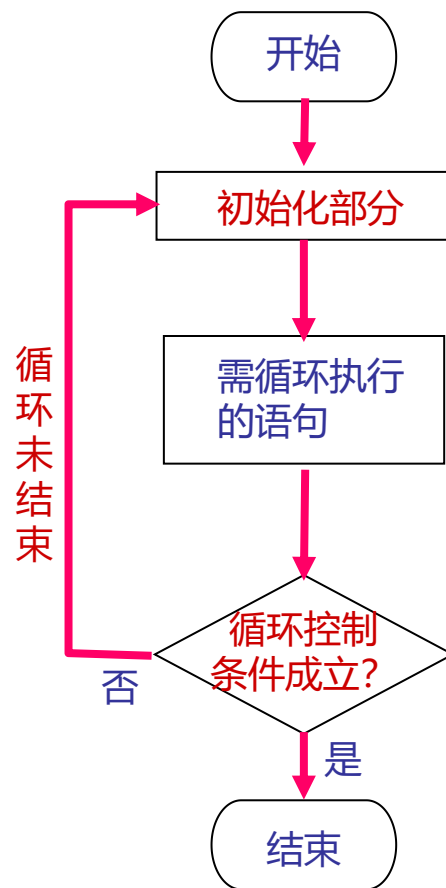


条件循环结构的流程图
(循环次数不确定)



训练题：关于右侧算法流程图是否有错误，你认为正确的选项是_____。

- ☐ A 无错误
- ☐ B 有1处错误
- ☐ C 有2处错误
- ☐ D 有3处错误
- ☐ E 有4处错误



提交

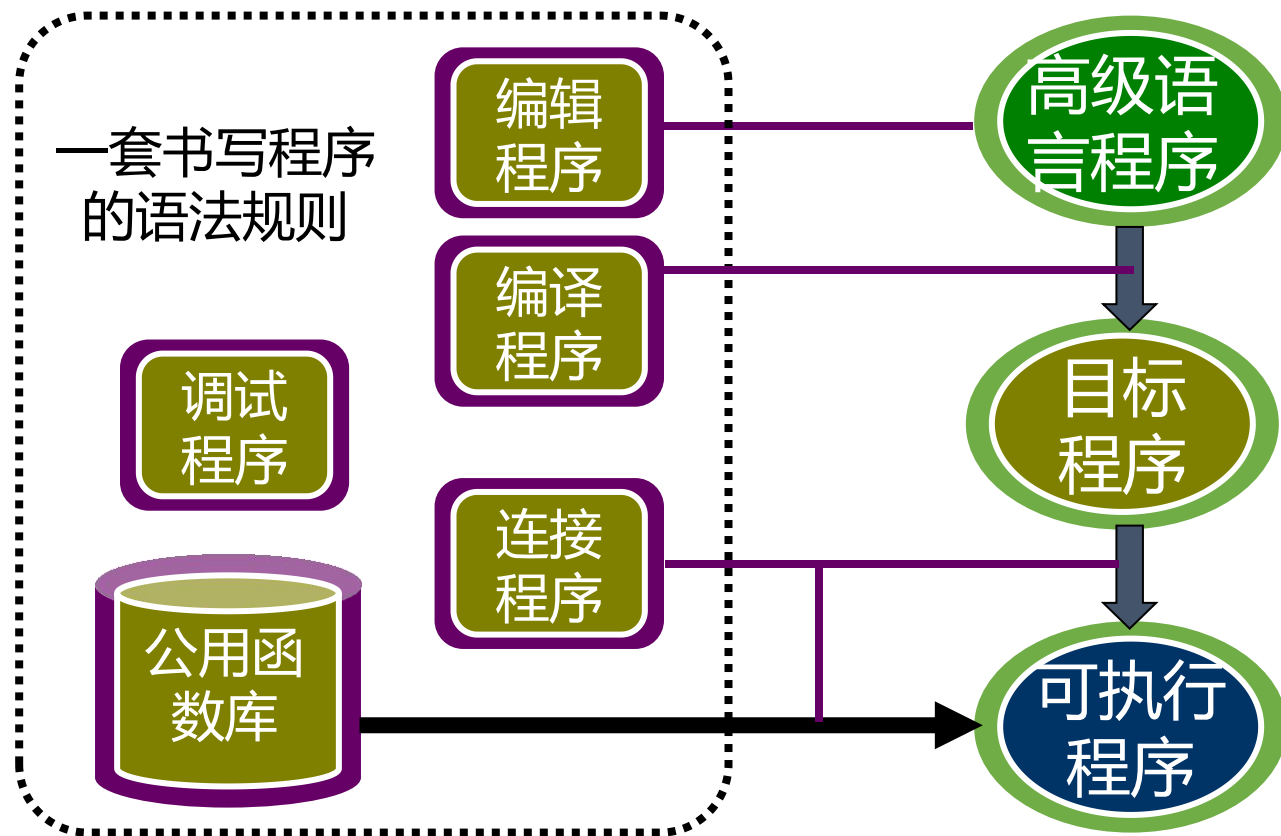
算法表达与思维

59

由算法到程序

程序设计过程：编辑源程序→编译→链接→执行。

计算机语言程序设计环境：编辑、编译、连接、调试、运行一体化平台



程序设计与思维小结

60

